## 一、单选题 (每题2分，共20分)

1. 80C51单片机复位后，堆栈指针SP的初始值是？ A. 00H B. 07H C. 08H D. 81H
2. 在MCS-51单片机中，哪个I/O端口用作通用I/O口时，必须外接上拉电阻？ A. P0 B. P1 C. P2 D. P3
3. 当`EA`引脚接高电平时，CPU访问的程序存储器范围是？ A. 仅片外0000H-FFFFH B. 仅片内0000H-0FFFH C. 先访问片内0000H-0FFFH，然后自动转向片外1000H-FFFFH D. 仅片外1000H-FFFFH
4. 下列哪条指令是专门用于访问程序存储器（ROM）的？ A. `MOV A, @R0` B. `MOVX A, @DPTR` C. `MOVC A, @A+DPTR` D. `MOV A, 50H`
5. 在乘法和除法指令中，除了累加器A，还必须使用哪个寄存器？ A. SP B. DPTR C. PSW D. B
6. `MOV A, @R0` 指令中，源操作数的寻址方式是？ A. 寄存器寻址 B. 直接寻址 C. 寄存器间接寻址 D. 立即寻址
7. 80C51的定时器T0工作在方式2下，其工作模式是？ A. 13位定时/计数器 B. 16位定时/计数器 C. 自动重装载的8位定时/计数器 D. 两个独立的8位定时/计数器
8. 在串行通信中，用于判断一帧数据是否发送完毕的标志位是？ A. RI (SCON.0) B. TI (SCON.1) C. REN (SCON.4) D. SM0 (SCON.7)
9. 下列关于 `LCALL` 指令的描述，哪个是错误的？ A. 它是一条三字节指令 B. 它会将断点地址压入堆栈 C. 它的调用范围是64KB D. 它不会改变SP寄存器的值
10. 关于定时器/计数器工作方式控制寄存器TMOD，下列说法正确的是？ A. 它可以进行位寻址 B. 它位于特殊功能寄存器区，地址为88H C. CPU复位后，其值为00H D. 它的高4位用于控制T0，低4位用于控制T1

## 二、判断题 (每题2分，共20分)

1. MCS-51单片机的所有指令周期都等于一个机器周期。( )
2. `MOV A, ACC` 这条指令是合法的，并且属于直接寻址。( )
3. 在中断优先级控制中，如果两个中断源的优先级相同，则它们永远不会相互打断。( )
4. 执行 `MOVX @DPTR, A` 指令时，单片机的 /RD 引脚会输出一个低电平脉冲。( )
5. `DEC DPTR` 是一条有效的指令，功能是将16位数据指针减1。( )
6. `DAA` 指令可以在任何算术运算指令之后使用，用于将结果调整为BCD码。( )
7. P3.0口的第二功能是串行数据发送（TXD）。( )
8. 当访问片外程序存储器时，`/PSEN` 信号有效（为低电平）。( )
9. MCS-51的堆栈是向上生长的，执行PUSH操作时，SP指针会加1。( )
10. `LJMP` 指令可以无条件转移到64KB程序存储器空间的任意地址。( )

## 三、填空题 (每空2分，共30分)

1. MCS-51单片机的4个I/O端口中，具有第二功能的是 ____ 端口。
2. 要选择工作寄存器组2，程序状态字寄存器PSW中的RS1应设置为 __，**RS0应设置为** __。
3. 一个机器周期包含 ____ 个状态，也就是 ____ 个振荡周期。
4. `CJNE A, #50H, LOOP` 指令的功能是比较A和立即数50H，若两者__**则转移。执行后若A < 50H，则进位标志CY为**__。
5. 定时器/计数器用作计数方式时，计数脉冲从其对应的外部引脚输入，T0的引脚是__，**T1的引脚是**__。
6. 80C51的特殊功能寄存器（SFR）中，地址可以被8整除的（即地址末位为0H或8H），通常都可以进行____寻址。

7. 要使能外部中断0和定时器1中断，并打开CPU总中断，IE寄存器的值应设置为____ (二进制形式)。
8. 串行口工作方式1的数据帧格式为10位，由1个__**位，8个**__位和1个__位组成。
9. 访问片外数据存储器，只能使用____指令。

## 四、程序填空题 (每空5分，共30分)

**题目要求：** 使用80C51单片机的定时器T0，工作在方式1（16位定时），实现在P1.0引脚上输出一个周期为2ms的方波。假设晶振频率为12MHz。请填写下面程序中的空白部分。

```
          ORG   0000H
          LJMP  MAIN
          ORG   000BH        ; T0 中断服务程序入口
          LJMP  T0_ISR

          ORG   0030H
MAIN:
          MOV   TMOD, #01H   ; 1. 设置T0为方式1
          MOV   TH0,  #____(1)____    ; 2. 装载定时初值高8位
          MOV   TL0,  #____(2)____    ; 3. 装载定时初值低8位
          SETB  TR0          ; 4. 启动定时器T0
          SETB  EA           ; 5. 开放CPU总中断
          ____(3)____         ; 6. 允许T0中断
LOOP:
          SJMP  LOOP         ; 7. 等待中断

T0_ISR:
          CLR   TR0          ; 8. 暂停T0，防止重装初值时出错
          MOV   TH0,  #____(4)____    ; 9. 重新装载初值
          MOV   TL0,  #____(5)____
          SETB  TR0          ; 10. 重启T0
          ____(6)____         ; 11. P1.0引脚电平取反
          RETI               ; 12. 中断返回
          END
```

## 一、单选题

1. **B. 07H**
   - **解析：** MCS-51单片机复位后，堆栈指针SP的内容被自动初始化为07H。这意味着第一个入栈操作（PUSH）的数据将存放在内部RAM的08H单元。
2. **A. P0**
   - **解析：** P0端口内部是漏极开路结构，没有内部上拉电阻。当它作为通用I/O口（特别是输出高电平）时，必须在外部连接上拉电阻才能正常输出高电平。
3. **C. 先访问片内0000H-0FFFH，然后自动转向片外1000H-FFFFH**
   - **解析：** `EA`是外部程序存储器访问允许引脚。当`EA`=1时，CPU首先从片内ROM的0000H开始执行程序，当PC值超出片内ROM的容量（如4K机型的0FFFH）时，会自动转向从片外的1000H地址继续取指令。
4. **C.** `**MOVC A, @A+DPTR**`
   - **解析：** `MOVC`指令中的 "C" 代表 "Code"，是专门用于从程序存储器（Code Memory）中读取数据的指令，通常用于查表操作。

5. **D. B**
   - **解析：** 在执行 `MUL AB` 乘法指令时，A和B寄存器为两个乘数，乘积的低8位在A中，高8位在B中。在执行 `DIV AB` 除法指令时，A为被除数，B为除数，商在A中，余数在B中。
6. **C. 寄存器间接寻址**
   - **解析：** 寄存器名称前加"@"符号是寄存器间接寻址的标志。该指令表示将寄存器R0中的内容作为地址，去片内RAM中寻址，然后将该地址单元的内容传送给累加器A。
7. **C. 自动重装载的8位定时/计数器**
   - **解析：** 方式2是8位自动重装载模式。TLx作为8位计数器，当计数溢出时，会自动将THx中的内容重新装载到TLx中，并置位TFx标志，申请中断。
8. **B. TI (SCON.1)**
   - **解析：** TI（Transmit Interrupt Flag）是发送中断标志位。当串行口成功发送完一帧数据后，硬件会自动将TI置1。
9. **D. 它不会改变SP寄存器的值**
   - **解析：** `LCALL`（长调用）指令会执行子程序调用操作，调用时需要将下一条指令的地址（断点）压入堆栈保存，这个过程会使SP的值加2。因此"不会改变SP寄存器的值"是错误的。
10. **C. CPU复位后，其值为00H**
    - **解析：** TMOD寄存器在CPU复位后会被清零，即所有定时器默认都处于方式0。A错误，TMOD不能位寻址；B错误，TMOD地址是89H；D错误，高4位用于T1，低4位用于T0。

## 二、判断题

1. **错误 ( × )**
   - **解析：** 指令周期可以包含1、2或4个机器周期，取决于指令的复杂性。
2. **正确 ( √ )**
   - **解析：** 访问SFR（如ACC）属于直接寻址方式。该指令功能上等同于 `MOV A，0E0H`。
3. **正确 ( √ )**
   - **解析：** 根据中断控制原则，正在执行的中断服务程序不能被同级别的中断请求所打断。
4. **错误 ( × )**
   - **解析：** `MOVX` 是访问外部数据存储器（RAM）的指令。写操作应使用写选通信号 `/WR`。因此，是 `/WR` 引脚输出低电平脉冲。
5. **错误 ( × )**
   - **解析：** 减1指令 `DEC` 的操作对象不能是16位的DPTR。`INC DPTR` 是有效的，但 `DEC DPTR` 不存在。
6. **错误 ( × )**
   - **解析：** `DAA`（十进制调整）指令必须紧跟在加法指令（如 `ADD`、`ADDC`）之后使用，用于将二进制和调整为压缩BCD码。
7. **错误 ( × )**
   - **解析：** P3.0口的第二功能是串行数据接收（RXD），P3.1的第二功能才是串行数据发送（TXD）。
8. **正确 ( √ )**
   - **解析：** `/PSEN`（Program Store Enable）是片外程序存储器的读选通信号，在CPU从外部ROM取指令时，该引脚输出低电平。

9. **正确 ( √ )**
    - **解析：** MCS-51的堆栈在内部RAM中由低地址向高地址生长，称为向上生长。执行PUSH操作时，SP先加1，再存入数据。
10. **正确 ( √ )**
    - **解析：** `LJMP` 是三字节长转移指令，其后的16位地址操作数可以直接指定64KB程序空间内的任何地址。

## 三、填空题

1. `P3`
2. `1`, `0`
3. `6`, `12`
4. `不相等`, `1`
5. `P3.4`, `P3.5`
6. `位`
7. `10001010B`
8. `起始` (低电平), `数据`, `停止` (高电平)
9. `MOVX`

## 四、程序填空题

**计算过程：**

1. **机器周期计算：**
    1. 晶振频率 fosc = 12MHz
    2. 机器周期 Tmachine=12/fosc=12/12MHz=1μs
2. **定时初值计算：**
    1. 
    2. 方波周期为2ms，所以半周期（定时时间）为1ms。
    3. 
    4. 需要计数的机器周期数 = 1ms/1μs=1000
    5. 
    6. 定时器初值 X = 216−1000=65536−1000=64536
    7. 
    8. 将64536转换为十六进制：`FC18H`
    9. 
    10. 因此，TH0 = `FCH`，TL0 = `18H`

**答案：**

1. `0FCH`
2. `018H`
3. `SETB ET0`

4. `0FCH`
5. `018H`
6. `CPL P1.0`

## 一、单选题 (每题2分，共20分)

1. 在80C51中，指令 `ANL` P1, #0FH 执行时，读取P1口数据的来源是？ A. P1端口的物理引脚 B. P1端口的输出锁存器 C. 累加器A D. 程序状态字PSW
2. 若80C51单片机在执行一条双周期指令（如 `LJMP`）的第一个周期时，一个高优先级的外部中断请求有效，CPU会在何时响应中断？ A. 立即暂停当前指令，响应中断 B. 执行完当前指令的第一个周期后响应中断 C. 执行完整个 `LJMP` 指令后响应中断 D. 忽略该中断请求
3. 在Cortex-M处理器中，读取程序计数器PC的值，得到的是 `当前指令地址 + 4`，这是由什么原因造成的？ A. 内存地址对齐的要求 B. 编译器的优化 C. 处理器的三级流水线结构 D. 为了兼容旧的ARM指令集
4. Cortex-M处理器中，`MOV R0, R1` 和 `MOVS R0, R1` 指令的主要区别在于？ A. `MOVS` 只能用于16位指令 B. `MOVS` 执行速度更快 C. `MOVS` 会根据传送的数据更新APSR中的N和Z标志位 D. `MOVS` 可以操作特殊功能寄存器
5. 80C51单片机响应中断后，硬件会自动完成一系列操作，但不包括以下哪一项？ A. 将PC内容压入堆栈 B. 根据中断源跳转到对应的中断向量地址 C. 将当前PSW内容压入堆栈 D. 封锁同级和低级中断
6. Cortex-M处理器支持非对齐内存访问，但这通常会导致什么后果？ A. 无法访问数据 B. 产生一个硬件错误（HardFault） C. 处理器需要执行多次总线操作，降低了访问效率 D. 必须在特殊模式下才能工作
7. 要将Cortex-M处理器的一个GPIO引脚（如PA5）设置为通用推挽输出模式，需要配置哪个寄存器？ A. `GPIOA->MODER` B. `GPIOA->OTYPER` C. `GPIOA->PUPDR` D. `GPIOA->BSRR`
8. 80C51的定时器工作在方式3时，定时器0（T0）被拆分为两个独立的8位定时器。此时，如果TH0溢出，它将置位哪个中断标志位？ A. TF0 B. TF1 C. 它不置位任何标志位 D. 同时置位TF0和TF1
9. 在Cortex-M的NVIC中，若两个中断的抢占优先级相同，但响应优先级不同，当它们同时触发时，系统会如何处理？ A. 响应中断向量地址较小的中断 B. 响应响应优先级较高的（即数值较小的）中断 C. 随机响应一个 D. 产生一个总线错误（BusFault）
10. 80C51的 `CJNE A, direct, rel` 指令执行后，若 `A` 的内容小于 `direct` 单元的内容，进位标志CY的状态是？ A. CY = 1 B. CY = 0 C. CY状态不变 D. CY被随机置位

## 二、判断题 (每题2分，共20分)

1. 80C51单片机的P0-P3端口在复位后都处于高电平状态，可以直接用作输入口读取外部电平。( )
2. 在Cortex-M中，执行 `PUSH {R0, R1}` 指令后，主堆栈指针MSP的值会减8，并且地址较低的内存单元存放的是R0的内容。( )
3. 80C51的变址寻址指令 `MOVC A, @A+PC`，其基址寄存器PC的值是该 `MOVC` 指令的下一条指令的地址。( )
4. Cortex-M处理器从地址0x00000000处获取MSP的初始值，从0x00000004处获取复位向量，该向量地址的最低位（bit0）必须为1。( )
5. 80C51串行口工作在方式1，要实现2400的波特率，若SMOD=1，其效果等同于晶振频率加倍后SMOD=0时的设置。( )
6. Cortex-M的堆栈模型是“向下生长的满栈”，意味着SP总是指向栈顶最后一个有效数据单元。( )
7. 80C51执行 `ACALL` 指令与响应中断时，都需要将断点地址压栈，这两个断点地址的含义是完全相同的。( )
8. 在STM32中，通过访问位段别名区的一个字地址，可以实现对位段区中一个特定比特位的“原子”读写操作。( )

9. 80C51中，指令 `XCH A, @R0` 和 `XCHD A, @R0` 都实现了累加器A和R0所指向的RAM单元之间的数据交换。( )
10. Cortex-M的链接寄存器LR（R14）只在发生函数调用（如BL指令）时才会被自动更新。( )

## 三、填空题 (每空2分，共20分)

1. 在Cortex-M处理器中，一个32位的字变量，如果其起始地址是 `0x20000006`，则这种访问称为 ____ 访问，处理器需要执行两次总线操作来读取它。
2. 在80C51中，若要实现对片内RAM高128字节（地址80H-FFH）的访问，必须采用 ____ 寻址方式。
3. Cortex-M的通用寄存器中，____ 到 ____ 被称为低寄存器，可以被所有的16位Thumb指令访问。
4. 80C51的定时器T1用作串行口波特率发生器时，最常用的工作方式是方式2，即____模式。
5. 在Cortex-M处理器的Thumb-2指令集中，`MOVW` 指令用于加载一个16位数到寄存器的__，**而 `MOVT` 指令用于加载一个16位数到寄存器的__**。
6. 80C51的P3口是准双向I/O口，当它被用作输入时，必须先向其端口锁存器写入 ____ ，以关断上拉的P-MOS管。
7. Cortex-M处理器响应一个中断时，硬件会自动压栈，保存包括R0-R3、R12、____、PC和xPSR在内的8个寄存器。
8. 80C51的PCON寄存器中的 ____ 位可以使串行通信的波特率加倍。

## 四、51单片机程序填空题 (每空1分，共30分)

**1. 查表求平方**

```
; 功能: 查找A寄存器中数值(0-9)的平方，结果存入B寄存器。
; 平方数表TABLE存放在程序存储器中。
LOOKUP_SQUARE:
MOVC   A, @A + (1)     ; 以PC做基址寄存器查表
MOV    B, A                    ; 将结果存入B
RET
TABLE:
DB     0, 1, 4, 9, 16, 25, 36, 49, 64, 81
```

**2.** 外部中断计数

```
; 功能: 使用外部中断0 (INT0) 对P3.2引脚上的负跳变进行计数。
; 计数值存放在内部RAM的30H单元。INT0设置为边沿触发。
ORG    0000H
LJMP   MAIN
ORG    0003H               ; INT0 中断向量地址
LJMP   INT0_ISR
MAIN:
MOV    30H, #0             ; 计数值清零
SETB   IT0                 ; (2) 设置INT0为边沿触发
SETB   EX0                 ; (3) 允许外部中断0
SETB   EA                  ; (4) 开放CPU总中断
SJMP   $                   ; 等待中断
INT0_ISR:
INC    (5)            ; 计数值加1
RETI                       ; 中断返回
```

### 3. 数据块清零

```
; 功能：将片外RAM从2000H到20FFH的256个字节清零。
MOV   DPTR, #(6)     ; 设置起始地址
MOV   R7, #0              ; 设置循环次数高位 (256次循环)
MOV   A, #0               ; 准备要写入的数据0
LOOP_CLR:
MOVX  (7), A         ; 向片外RAM写入0
INC   (8)            ; 地址指针加1
DJNZ  R7, LOOP_CLR        ; R7不为0则继续，但DJNZ只对8位寄存器操作
; 这里用R7=0实现256次循环的技巧
SJMP  $
```

### 4. 串行口发送单个字符'Y'

```
; 功能：设置串行口为方式1，波特率由T1方式2自动重装载产生。
; 晶振11.0592MHz，波特率9600。发送字符'Y'。
; T1初值计算：TH1 = 256 - (11.0592MHz / 12 / 32 / 9600) = 253 = 0FDH
MOV   TMOD, #(9)     ; 设置T1为方式2
MOV   TH1, #(10)    ; 装载波特率初值
SETB  TR1                 ; (11) 启动定时器T1
MOV   SCON, #(12)    ; 设置串行口为方式1，允许发送
MOV   SBUF, #(13)    ; 将字符'Y'的ASCII码送入发送缓冲器
WAIT_TI:
JNB   TI, WAIT_TI         ; (14) 等待发送完成
CLR   (15)           ; 清除发送完成标志
SJMP  $
```

### 5. 寻找数据块最大值

```
; 功能：在内部RAM 40H-4FH的16个字节数据中，找到最大值，并存放到50H单元。
MOV   R0, #40H            ; (16) 设置数据块起始地址指针
MOV   R7, #(17)      ; 设置循环次数
MOV   A, @R0              ; (18) 先取第一个数作为当前最大值
LOOP_MAX:
INC   R0                  ; (19) 指向下一个数
MOV   B, A                ; (20) 暂存当前最大值
SUBB  A, @R0              ; (21) 用减法比较 (A - @R0)，结果不保存
MOV   A, B                ; (22) 恢复当前最大值
JNC   NO_UPDATE           ; (23) 如果A >= @R0 (无借位)，则不更新
MOV   A, @R0              ; (24) 如果A < @R0，则更新最大值
NO_UPDATE:
DJNZ  R7, LOOP_MAX        ; (25) 循环比较下一个数
MOV   (26), A        ; (27) 将最终找到的最大值存入50H
```

### 6. 软件延时

```
; 功能：一个简单的双重循环延时子程序。
DELAY:
MOV   R6, #100          ; (28) 外层循环计数
L1: MOV   R5, #200          ; (29) 内层循环计数
L2: DJNZ  R5, L2            ; 内层循环
DJNZ  (30), L1        ; 外
```

## 一、单选题

1. **B. P1端口的输出锁存器**
   - **解析：** `ANL P1, #0FH` 是一条“读-修改-写”指令。这类指令在执行时，为了避免因引脚电平变化（如外部负载影响）导致操作错误，它读取的是端口内部锁存器的状态，而不是引脚的实际电平。修改后的值再同时写回锁存器和引脚。
2. **C. 执行完整个 `**LJMP**` 指令后响应中断**
   - **解析：** CPU响应中断的条件之一是当前指令必须执行完毕。即使是多周期指令，也必须等待其所有机器周期全部完成，CPU才会去查询中断标志并响应中断。
3. **C. 处理器的三级流水线结构**
   - **解析：** Cortex-M采用“取指-译码-执行”三级流水线。当CPU正在执行当前指令时，它已经完成了下一条指令的取指和译码，并且正在取下下一条指令。因此，PC总是指向“正在取指”的指令地址，即 `当前执行指令地址 + 4`。
4. **C. `**MOVS**` 会根据传送的数据更新APSR中的N和Z标志位**
   - **解析：** 指令后缀 `S` 的作用就是使该指令在执行后更新APSR（应用PSR）中的条件标志位，如N（负）、Z（零）、C（进位）、V（溢出）。`MOV` 指令则不影响这些标志位。
5. **C. 将当前PSW内容压入堆栈**
   - **解析：** 80C51硬件中断响应过程只会将PC（断点地址）压栈。保护现场（如保存PSW、ACC等关键寄存器）需要用户在中断服务程序的开头用 `PUSH` 指令自行完成。
6. **C. 处理器需要执行多次总线操作，降低了访问效率**
   - **解析：** Cortex-M的硬件可以处理非对齐访问，它会将一次非对齐的访问拆分成两次（或更多次）对齐的访问，然后将结果在内部拼接起来。这个过程对软件是透明的，但会消耗额外的时钟周期，导致性能下降。
7. **A. `**GPIOA->MODER**`**
   - **解析：** GPIOx_MODER（端口模式寄存器）**用于配置每个引脚的I/O方向模式（输入、输出、复用、模拟）**。要设置为输出模式，需要修改此寄存器中对应的位。`OTYPER`设置输出类型（推挽/开漏），`PUPDR`设置上下拉，`**BSRR**`**用于置位/复位引脚**。
8. **B. TF1**
   - **解析：** 这是方式3的一个特殊之处。当T0工作在方式3时，TL0作为一个8位定时器，使用T0的所有控制位（TR0、TF0、INT0中断）；而TH0也作为一个独立的8位定时器，但它“借用”了T1的控制位TR1和中断标志TF1。因此TH0溢出时，置位的是TF1。
9. **B. 响应响应优先级较高的（即数值较小的）中断**
   - **解析：** NVIC的优先级仲裁原则是：首先比较抢占优先级，抢占优先级高的胜出。如果抢占优先级相同，则比较响应优先级，响应优先级高的（数值小）胜出。
10. **A. CY = 1**

- **解析：** `CJNE` 指令内部实际上是执行了一个减法操作来比较。格式 `CJNE A, data, rel`，硬件计算 `A - data`。如果发生借位（即 `A < data`），则CY=1。如果A >= data，则CY=0。

## 二、判断题

1. **错误 ( × )**
   - **解析：** P1-P3复位后为高电平，可直接用作输入。但P0口内部是漏极开路的，复位后处于高阻态，不是高电平。必须外接上拉电阻才能稳定地读到高电平。
2. **错误 ( × )**
   - **解析：** Cortex-M的堆栈是向下生长的，SP指针会减小。`PUSH` 操作后，SP值会减8。 在Cortex-M中，当使用 `PUSH` 或 `POP` 一次性操作多个寄存器时，**寄存器在内存中的存取顺序取决于寄存器的编号（编号高的先入栈），而与您在指令 `**{}**` 中书写它们的顺序完全无关。**
3. **正确 ( √ )**
   - **解析：** PC在取指令后会自动加1（或指令字节数）。当CPU执行`MOVC A, @A+PC`时，PC的值已经指向了该条指令的下一条指令的地址。
4. **正确 ( √ )**
   - **解析：** 这是Cortex-M的复位序列。向量地址的最低位用于指示处理器应切换到Thumb状态执行，因此必须为1。
5. **正确 ( √ )**
   - **解析：** 波特率的计算公式与 2SMOD 和 fosc 有关。将SMOD置1，波特率加倍；将晶振频率加倍，波特率也加倍。因此效果是等同的。
6. **正确 ( √ )**
   - **解析：** 向下生长（Full-Descending）指堆栈由高地址向低地址扩展，满栈（Full）指SP指针总是指向栈顶最后一个被压入的数据。
7. **错误 ( × )**
   - **解析：** `ACALL`压栈的断点是`ACALL`指令之后的那条指令的地址。而中断响应时压栈的断点是当前被中断指令的下一条指令的地址。虽然都是“下一条指令”，但触发的时机和上下文不同，不能认为完全相同。
8. **正确 ( √ )**
   - **解析：** 这是位段（Bit-band）操作的核心原理。对别名区的字访问被硬件自动映射为对位段区目标比特的读-修改-写操作，这个过程不会被中断打断，因此是原子的。
9. **错误 ( × )**
   - **解析：** `XCH A, @R0` 交换A和RAM单元的整个字节。而`XCHD A, @R0` 只交换A的低4位和RAM单元的低4位，高4位保持不变。
10. **错误 ( × )**
    - **解析：** 除了函数调用，当发生中断或异常时，硬件也会自动将特殊的返回信息（如EXC_RETURN）加载到LR寄存器中，用于中断返回。

## 三、填空题

1. `非对齐`
2. `寄存器间接`
3. `R0`，`R7`

4. `8位自动重装载`
5. `低16位`，`高16位`
6. `1`（或 `0FFH`）
7. `LR`（或 R14）
8. `SMOD`

## 四、51单片机程序填空题

1. `(1) PC`
2. `(2) SETB IT0`，`(3) SETB EX0`，`(4) SETB EA`，`(5) 30H`
3. `(6) 2000H`，`(7) @DPTR`，`(8) DPTR`
4. `(9) 20H`，`(10) 0FDH`，`(11) SETB TR1`，`(12) 40H`，`(13) 'Y'`，`(14) JNB TI, WAIT_TI`，`(15) TI`
5. `(16) MOV R0, #40H`，`(17) 0FH`（循环15次，因为第一个数已读），`(18) MOV A, @R0`，`(19) INC R0`，`(20) MOV B, A`，`(21) SUBB A, @R0`，`(22) MOV A, B`，`(23) JNC NO_UPDATE`，`(24) MOV A, @R0`，`(25) DJNZ R7, LOOP_MAX`，`(26) MOV 50H, A`
6. `(28) MOV R6, #100`，`(29) MOV R5, #200`，`(30) R6`

## 判断题

1、 P0 口的第一功能是准双向 I/O 口，第二功能是分时复用的低 8 位地址线和 8 位数据线。

2、 CPU 响应中断时，硬件自动进行断点地址保护，自动转去执行中断服务程序。

3、 中断服务程序的保护现场也是单片机硬件自动完成的。

4、 中断服务程序的最后一条指令是 RET。

5、 MOVC A,@A+DPTR 是一条查表指令，寻址空间是 ROM。

6、 MUL AB 的执行结果是高 8 位在 A 中，低 8 位在 B 中。

7、 DPTR 是由 DPH 和 DPL 两个 8 位特殊寄存器组成的。

8、 由于 MCS-51 串行口的数据发送和接收缓冲器都是 SBUF，所以其串行口不能同时发送和接收数据，即不是全双工的串行口。

9、 ADC0809 是 8 位逐次逼近式模/数转换器。

10、 51 单片机复位后，堆栈指针 SP 中的内容为 07H。

11、 51 单片机的串行口有四种工作方式。

12、 51 单片机串行口的工作方式 0 是 I/O 扩展方式。

13、 定时器/计数器 T0、T1 工作在计数方式时，能够计数任意频率的外部脉冲。

14、 MCS-51 单片机的各中断源发出中断请求时，都会在 TCON 中建立相应的标志。

15、 只有十进制加法和减法运算，才能进行十进制调整。

16、 执行返回指令时，返回的断点是此时堆栈顶部的内容。

17、 CPU 和外设进行数据传送时主要采用查询方式和中断方式，两者相比后者能节省 CPU 资源。

18、 MCS-51 单片机的五个中断源中，优先级最高的是外部中断 0，优先级最低的是串行口中断。

19、 MCS-51 单片机的五个中断源有五个优先级。

20、 0 区工作寄存器的 R2 所对应的内部存储器的地址是 03H 。

21、 8051 单片机的 P2 口只能用作通用 I/O 口。

22、 程序存储器和数据存储器的作用不同，程序存储器一般用来存放数据表格和程序，而数据存储器用来存放数据。

23、 DA A 指令只能用在 BCD 运算的加法指令后，才能起到十进制调整的作用。

24、 MCS-51 单片机中的特殊功能寄存器，可采用寄存器间接寻址方式。

25、 MCS-51 系列单片机中，高级中断能够打断低级和同级中断。

26、 若 A 中的内容为 23H，那么，PSW 中的 P 标志位为__1___。

27、 执行指令 MOV SBUF，A 能够启动 MCS-51 系列单片机的串行发送。

28、 MOV SP，＃5FH 开辟的堆栈空间为内部 RAM 的 60H 单元开始。

29、 MCS--51 单片机是 8 位机，但可以进行 16 位运算。

30、 NOP 不会使计算机产生任何操作，因此属于伪指令。

31、 在 MCS-51 中，为使准双向的 I/O 口工作在输入方式，必须先预置为 1。

32、 MOVX A,@DPTR 是向外部 RAM 写一个字节数据。

33、 汇编语言源程序中的伪指令汇编成目标程序时不产生机器码。

34、 访问片内 RAM 用 MOV 指令，访问片外 RAM 用 MOVX 指令。

35、 位地址 98H，位于 8031 的 20H～2FH 的位寻址区中。

36、 MCS-51 单片机启动串行发送的指令是 MOV SBUF，A ，启动串行接收的指令是 MOV A，SBUF。

37、 END 表示指令执行到此结束。

38、 定时器与计数器的工作原理均是对输入脉冲进行计数。

39、 TMOD 中 GATE=1 时，表示 INT0 或 INT1 参与控制定时器的启停。

40、 相对寻址方式中，“相对”两字是相对于当前指令的首地址。

41、 要进行多机通信，MCS-51 串行接口的工作方式应为方式 1。

42、 MCS-51 系统对串行口接收中断标志 RI 只能用软件进行清零。

43、 共阴极数码管能够显示数码的必要条件是其共同端接高电平。

44、 JBC P1.0, rel 和 JB P1.0, rel 均为转移指令，但它们的转移条件不同。

45、 MCS-51 系列芯片特殊功能寄存器区按字节寻址时可以采用间接寻址。

46、 8051 单片机的算术逻辑部件能完成带借位和不带借位的减法运算。

47、 串行通讯接收到的第 9 位数据送 SCON 寄存器的 RB8 中保存。

48、 MCS-51 的定时器/计数器对外部脉冲进行计数时，要求输入的计数脉冲的高电平和低电平的持续时间均不小于 1 个机器周期。

49、 MCS-51 中工作寄存器 R7 的实际物理地址与 PSW 中内容有关。

50、 MCS-51 在串行方式 0 时 TXD 是时钟发送端，RXD 是数据接收端。

## 填空题

1. 8051 的引脚 RST 是_____引脚，当其出现_____电平时，8051 进入复位状态直到 RST 引脚变为_____电平，8051 才脱离复位状态，进入程序运行状态，从 ROM_____ H 单元开始取指令并翻译和执行。

2. 半导体存储器分成两大类_____和 _____，其中_____具有易失性，常用于存储 _____ 。

3. 8051 内部有_____个并行口，P0 口直接作输出口时，必须外接_____；这些端口作输入口时，必须先_____ ，才能正确读入外设的状态。

4. MCS-51 的堆栈只可设置在_____ ，其最大容量为_____，存取数据的原则是_____。堆栈寄存器 SP 是_____位寄存器，存放_____ 。

5. 定时和计数都是对_____ 进行计数，定时与计数的区别是 _____。

6. MCS—51 单片机外部中断有低电平和_____ 两种触发方式，在电平方式下，当采集到 INT0、INT1 的有效信号为_____时，触发外部中断。

7. 单片机内有组成微型计算机的主要功能部件_____、_____和_____。

8. MCS-51 的指令按其功能可分为五大类：_____、_____、逻辑运算类指令、位操作指令、控制转移指令。

9. CJNE <目的字节>，<源字节>，rel 属于五大类指令中的_____类指令，其操作码助记符含义是_____。

10. MCS-51 的 P0 口作为输出端口时，每位能驱动___个 TTL 负载。

11. 单片机中断系统中共有 _____、 _____、 _____、 _____、_____五个中断源；当它们处于同一优先级并同时申请中断时，最优先得到响应的中断是_____。

12. 在 MCS—51 单片机内部 RAM 中， 20H～2FH 存储空间为_____ ，30H～7FH 存储空间为_____。

13. 计算机系统或单片机都是通过_____、_____、_____三总线连接内部各功能模块的。

14. 运算器是山_____、_____和_____等几部分组成，用来执行各种算术运算和逻辑运算。

15. MCS-51 的堆栈，可以通过软件修改堆栈指针的内容在_____内开辟的一块存储空间。

16. MCS-51 有_____个工作寄存器区，它们的地址范围是_____。

17. DJNZ <操作数>，rel 属于五大类指令中的_____类指令，其操作码助记符含义是_____。

18. 微处理机的寻址能力（范围）山_____决定。若某单片机有 14 根地址线，则它可寻址的存储器范围为_____KB。

19. MCS--51 单片机中，通用内部 RAM 中的位寻址区是_____，它们的位地址范围是 _____。

20. 程序计数器指针是_____，堆栈指针是_____，数据指针是_____。

21. 外设（输入输出设备）与主机（或单片机）必须通过_____进行信息传送，输入口必须具有_____功能，输出口必须具有_____功能。

22. P0、P1、P2、P3 四个均是_____位的_____口，其中 P0 的第二功能是_____；P2 口的第二功能是_____。

23. 8051 单片机片外程序存储器的读选通信号是_____，数据存储器的读选通信号是____。

24. MCS－51 单片机有 7 种寻址方式，分别是：_____寻址、_____寻址、_____寻址、_____寻址、_____寻址、_____寻址及_____寻址。

25. 循环结构程序中循环控制的实现方法有_____和_____。

26. 如果系统晶振频率为 6MHz，则定时器/计数器工作方式 1 的最大定时时间为_____，若系统晶振频率为 12MHz，则最大定时时间为_____。

27. 定时时间与定时器的_____、_____及_____有关。

28. SCON 的 SM2 是多机通信控制位，若置 SM2=1，则允许多机通信。多机通信时串行口应用于方式_____和方式_____。

29. 若 8 位 D/A 转换器的输出满刻度电压为+5V，则该 D/A 转换器能分辨的最小电压变化为_____，当输出数字量为 58H 时，对应的输出电压为 _____。

30. 若 ADC0809 $U_{REF}$=5V，输入模拟信号电压为 2.5V 时，A/D 转换后的数字量是_____，若 A/D 转换后的结果为 60H，输入的模拟信号电压为_____。

## 选择题

1、 存储器的地址范围是 0000H~0FFFH，它的容量为（　　）。

（A）1KB　　(B) 2KB　　(C) 3KB　　(D)　4KB

2、 若（A）=86H，（PSW）=80H，则执行 RRC　A 指令后（　）

（A）C3H　　(B) B3H　　(C) 0DH　　(D) 56H

3、 如果（P0）=65H ，则当

```
CPL    P0.2
SETB   C
MOV    P0.4 ，C
```

执行后（P0）=（　　）

（A）61H　　(B) 75H　　(C) 71H　　(D) 17H

4、 (　　)指令是 MCS-51 指令系统中，执行时间最长的指令。

（A）比较转移　（B）循环转移　（C）增减量　（D）乘除法

5、 8031 单片机中，将累加器 A 中的数送入外部 RAM 的 40H 单元，可执行的指令（　　）。

(A)　MOV　R1 ，40H
　　MOVX　@R1 ，　A

(B)　MOV　R1，#40H
　　MOVX　R1，　A

(C)　MOVX　40H ，　A

(D)　MOV　R1，　#40H
　　MOVX　@R1 ，　A

6、 已知累加器 A 的数值为 98H，它与 0FAH 相加，相加后会影响标志位 CY、AC 和 OV 位。各个位的值是（　　）。

(A) 0、0、0　　(B) 1、0、1　　(C) 1、1、1　　(D) 1、1、0

7、 A/D 转换方法有以下四种，ADC0809 是一种采用（　）进行 A/D 转换的 8 位接口芯片。

(A) 计数式　　(B) 双积分式　　(C) 逐次逼近式　　(D) 并行式

8、 下列指令写法正确的是（　　）。

（A）MOV　#20H，A　（B）ADD R2，#30H　（C）PUSH　R0　（D）RRC　A

9、 中断服务程序中至少应有一条（ ）。

A、传送指令 B、转移指令 C、加法指令 D、中断返回指令

10、 执行 LCALL 4000H 指令时， MCS-51 所完成的操作是( )。

A、保护 PC B、4000H→PC C、保护现场 D、PC+3 入栈， 4000H→PC

11、 下面哪条指令产生$\overline{WR}$信号( )。

A、MOVX A，@DPTR B、MOVC A，@A+PC C、MOVC A，@A+DPTR

D、MOVX @DPTR，A

12、 若某存储器芯片地址线为 12 根，那么它的存储容量为( )。

A、 1KB B、 2KB C、4KB D、8KB

13、 若 RS1=1，RS0=1，则当前工作寄存器是( )。

A、0 组 B、 1 组 C、 2 组 D、 3 组

14、 MCS-51 有中断源( )。

A、5 B、 2 C、 3 D、 6

15、 执行 PUSH ACC 指令， MCS-51 完成的操作是( )。

A、（SP）+1→（SP），（A）→((SP)) B、（A）→((SP))，（SP）－1→（SP）

C、（SP）－1→（SP），（A）→((SP)) D、（A）→((SP))，（SP）+1→（SP）

16、 P1 口的每一位能驱动( )。

A、2 个 TTL 低电平负载 B、 4 个 TTL 低电平负载

C、8 个 TTL 低电平负载 D、10 个 TTL 低电平负载

17、 PC 中存放的是( )。

A、下一条指令的地址 B、 当前正在执行的指令

C、当前正在执行指令的地址 D、下一条要执行的指令

18、 要把 P0 口高 4 位变 0，低 4 位不变，应使用指令( )。

A、ORL P0，#0FH B、ORL P0，#0F0H C、ANL P0，#0F0H D、ANL P0，#0FH

19、 计算机在使用中断方式与外设交换信息时，保护现场的工作应该是(　　)。

A、山 CPU 自动完成　B、在中断响应中完成　C、应山中断服务程序完成 D、在主程序中完成

20、 MCS-51 系统中，若晶振频率为 12MHz，一个机器周期等于(　　) μs。

A、1.5　　B、 3　　C、 1　　D、 0.5

21、 十进制 29 的二进制表示为原码（　　）。

A 、11100010　　B 、10101111　　C、00011101　　D、 00001111

22、 用 MCS-51 的串口扩展并行 I/O 口时，串行接口工作方式选择（　　）。

A 、方式 0　B、方式 1　　C、方式 2　　D、方式 3

23、 MCS—51 单片机外部有 40 个引脚，其中地址锁存允许控制信号引脚是（　　）。

A 、RST　　B、$\overline{PSEN}$　　C、$\overline{EA}$　　D、ALE

24、 对程序存储器的读操作，只能使用（　　）。

A 、MOV 指令　　B、PUSH 指令　C、MOVX 指令　　D、MOVC 指令

25、 MCS-51 单片机在同一优先级的中断源同时申请中断时，CPU 首先响应(　　)。

A、外部中断 0　　B、外部中断 1

C、定时器 0 中断　　D、定时器 1 中断

26、 执行中断返回指令，从堆栈弹出地址送给（　　）。

A、 A　　B、 CY　　C、 PC　　D、 DPTR

27、 执行指令 CLR　30H 后，结果被清 0 的是：_____。

（A）30H 的最低位　（B）24H 的最高位　（C）30H 单元　　（D）26H 单元的第 0 位

28、 复位使 MCS51 单片机进行自身初始化操作，它使专用寄存器进入各自的复位初态，其中 SP、PSW、P1 的状态为_____。

（A）00H，00H，00H　　（B）00H，00H，0FFH

（C）07H，00H，0FFH　　（D）07H，0FFH，00H

29、 要把 P0 口高 4 位不变, 低 4 位求反, 应使用指令_____。

（A）XOR P0, #0F0H　　（B）XOR P0, #0FH

（C）XRL P0, #0FH　　（D）XRL P0, #0F0H

30、 下列指令中影响标志位 C 的指令有_____。

（A）DA A　　（B）CLR A　　（C）RR A　　（D）INC A

31、 8031 单片机在执行 MOVC A, @A+PC 指令时，将发出地址信号和 _____信号。

（A）RD　　（B）PSEN

（C）RD 及 WR　　（D）RD 及 PSEN

32、 假定（SP）=62H，（60H）=50H，（61H）=40H，（62H）=30H，（63H）=20H，执行 RET 指令后，PC 的值为：_____；

（A）3020H　　（B）3040H　　（C）4030H　　（D）4050H

33、 欲将累加器中的高、低四位进行交换，应该选用的指令是：_____。

（A）XCH；　　（B）XCHD；　　（C）SWAP；　　（D）RLC；

34、 执行子程序返回指令时，从堆栈顶部弹出的内容送给_____。

（A） A　（B） CY　（C）PC　（D） DPTR

35、 若标号 LABEL 所在地址为 1040H，则地址 1000H 处的指令 SJMP LABEL 的转移偏移量为_____。

（A）3EH　　(B) 42H　　(C) 40H　　(D) 0E0H

36、 8031 单片机中既可位寻址又可字节寻址的单元是_____。

（A）28H　　（B）30H　　（C）08H　　（D）70H

37、 复位后执行 PUSH 07H，是把：_____。

（A）07H 单元内容压入 08H 单元　　（B） 08H 单元内容压入 07H 单元

（C）07H 单元内容压入 07H 单元　　（D） 错误指令

38、 在 CPU 内部，反映程序运行状态或反映运算结果特征的寄存器是_____；

（A）PC　（B）PSW　（C）A　（D）SP

39、 若（A）=68H，执行 XRL A，#98H 后，PSW 中被改变的标志有_____；

（A）没有； （B）P 标志； （C）C，AC，OV 标志； （D）全部

40、 在串行通信中， 8031 中发送和接收的寄存器是_____。

（A）TMOD （B） SBUF （C） SCON （D）DPTR

41、 73.5 转换成十六进制数是_____。

（A）94.8H （B）49.8H （C）111H （D）49H

42、 下面哪条指令，单片机将发出 $\overline{\text{PSEN}}$ 信号_____。

（A）MOVX A,@DPTR（B）MOVC A,@A+PC

（C）JMP @A+DPTR （D）MOVX @DPTR,A

43、 80C51 的 P1 口用作输入口时，执行 MOV P1, #0FFH 后再执行 MOV A, P1 指令，累加器 A 中的内容为_____。

（A）00H （B）11H （C）FFH （D）不确定

44、 JZ e 的操作码地址为 1000H，e=20H，它转移的目标地址为_____。

（A）1022 H （B）1020H （C）1000H （D）101EH

45、 MCS--51 单片机在执行 MOVX A, @DPTR 指令时，将发出_____信号。

（A）地址信号和 RD 信号; （B）数据信号和 RD 信号;

（C）地址信号和 WR 信号; （D）数据信号和 WR 信号;

46、 MCS-51 的定时器/计数器应用时的位数是_____。

（A）8 位的； （B）16 位的； （C）24 位的； （D）根据方式确定位数；

47、 下列哪类指令没有操作 MCS-51 的堆栈区：_____。

（A）ACALL （B）ADD （C）PUSH （D）RET

48、 设某 8 位 A / D 转换器，满量程输入电压为 5 V，当输入电压为 3.4 5 V 时，其输出数字量_____。

（A）176　　（B）255　　（C）88　　（D）76

49、设某 10 位 D / A 转换器，满量程输入电压为 5 V，此转换器的分辨率为_____mV。

（A）1.22　　（B）2.44　　（C）4.88　　（D）19.6

50、以下所列的伪指令标识符写法中，写法正确的一组是_____。

（A）EQU，RETI　　（B） NOP,　DB

（C）DW,　ORG　　（D）DA，　END

1. 关于Cortex-M处理器的基本特性，下列描述错误的是？ A. 采用32位RISC架构 B. 寄存器和内部数据通路都是32位的 C. 采用32位寻址，最大地址空间为4GB D. 只能处理32位的字(word)数据

1. Cortex-M4处理器默认采用的数据存储顺序是？ A. 大端模式 (Big-Endian) B. 小端模式 (Little-Endian) C. 由编译器决定 D. 可在运行时任意切换

1. 在Cortex-M中，为了保证数据访问效率，一个半字(Half-word)变量的存放地址应该是？ A. 任意地址 B. 2的整数倍 C. 4的整数倍 D. 8的整数倍

1. Cortex-M3/M4处理器的指令流水线是几级？ A. 二级 B. 三级 C. 四级 D. 五级

1. Cortex-M处理器支持的Thumb-2指令集，其指令长度是？ A. 只能是16位 B. 只能是32位 C. 同时支持16位和32位指令 D. 只能是8位

1. 在GPIO的复位状态上，Cortex-M与8051的主要区别是？ A. Cortex-M为高电平，8051为高阻态 B. Cortex-M为低电平，8051为高电平 C. Cortex-M为高阻态，8051为高电平 D. 两者都是高电平

1. Cortex-M处理器采用了哪种计算机体系结构？ A. 普林斯顿结构 B. 改进型哈佛结构 C. 典型的冯·诺依曼结构 D. 纯粹的哈佛结构

1. 关于Cortex-M的通用寄存器，下列描述正确的是？ A. R0-R12都可以被所有16位指令访问 B. R0-R7被称为高寄存器 C. R8-R12被称为低寄存器 D. 一些16位指令只能访问低寄存器R0-R7

1. 在Cortex-M处理器中，R13寄存器的专用功能是？ A. 程序计数器 (PC) B. 链接寄存器 (LR) C. 栈指针 (SP) D. 通用数据寄存器

1. 在未使用操作系统的情况下，Cortex-M处理器通常使用哪个栈指针？ A. 主栈指针MSP B. 进程栈指针PSP C. MSP和PSP交替使用 D. 由用户程序指定任意寄存器

1. 当Cortex-M处理器执行一个函数或子程序调用指令（如BL）时，返回地址会自动保存在哪个寄存器中？ A. R13 (SP) B. R14 (LR) C. R15 (PC) D. 堆栈中

1. 关于Cortex-M中的程序计数器PC (R15)，下列说法与8051相比最显著的区别是？ A. PC是16位的 B. PC是只读的 C. PC是可读可写的 D. PC不存放下一条指令的地址

1. 在Cortex-M中，读取PC寄存器的值，得到的是？ A. 当前正在执行指令的地址 B. 下一条将要执行指令的地址 C. 当前指令地址加4 D. 当前指令地址减2

1. Cortex-M的程序状态寄存器xPSR不包含哪个部分？ A. 应用PSR (APSR) B. 中断PSR (IPSR) C. 执行PSR (EPSR) D. 数据PSR (DPSR)

1. APSR中的哪个标志位用于指示ALU运算结果为负数？ A. Z (零标志) B. C (进位标志) C. N (负数标志) D. V (溢出标志)

1. 在Cortex-M中，哪个寄存器保存了当前正在服务的中断号？ A. APSR B. IPSR C. EPSR D. LR

1. EPSR中的T位如果被清零，将会发生什么？ A. 处理器切换到ARM状态 B. 处理器进入休眠模式 C. 引起异常错误 D. 无任何影响

1. Cortex-M的堆栈模型是？ A. 向上生长的空栈 B. 向上生长的满栈 C. 向下生长的空栈 D. 向下生长的满栈

1. 在Cortex-M的堆栈操作中，执行一次PUSH操作，SP指针会如何变化？ A. 先存入数据，SP再自增4 B. SP先自增4，再存入数据 C. 先存入数据，SP再自减4 D. SP先自减4，再存入数据

1. 由于对齐要求，Cortex-M的堆栈指针SP的最低两位永远是？ A. 00 B. 01 C. 10 D. 11

1. Cortex-M的存储器映像中，通常用于存放程序的区域是？ A. SRAM区 B. 外设区 C. CODE区 D. 系统控制空间(SCS)

1. Cortex-M的存储器保护单元(MPU)的主要作用是？ A. 加快存储器访问速度 B. 定义各存储区域的访问权限，防止越界访问 C. 对存储器数据进行加密 D. 减少系统功耗

1. Cortex-M处理器复位后，首先执行的操作是？ A. 从0x00000004地址处读取复位向量 B. 跳转到主函数main() C. 从0x00000000地址处读取MSP的初始值 D. 清空所有通用寄存器

1. Cortex-M的向量表中，每个向量（地址）占多少位？ A. 8位 B. 16位 C. 32位 D. 64位

1. 向量表中的异常服务例程地址，其最低位必须为1，这是为了指示什么？ A. 这是一个中断服务例程 B. 这是一个高优先级异常 C. 处理器处于Thumb状态 D. 这是一个安全区的地址

1. 嵌套向量中断控制器(NVIC)不具备以下哪个功能？ A. 处理中断的优先级配置 B. 实现中断的嵌套（抢占） C. 管理中断的屏蔽 D. 直接执行中断服务程序中的算术运算

1. 在NVIC中，如果两个中断的抢占优先级相同，响应优先级不同，当它们同时到达时，CPU会如何响应？ A. 先响应中断号较小的 B. 先响应响应优先级较高的 C. 随机响应一个 D. 产生硬件错误

1. 在NVIC的优先级设置中，数值与其优先级的关系是？ A. 编号越大，优先级越高 B. 编号越小，优先级越高 C. 编号奇偶决定优先级 D. 编号与优先级无关

1. 在Cortex-M中，下列哪个属于内核异常，而不是外部中断(IRQ)？ A. GPIO引脚中断 B. 定时器溢出中断 C. SysTick中断 D. DMA传输完成中断

1. Cortex-M发生中断响应时，硬件自动压栈的寄存器不包括？ A. R0-R3 B. R12 C. R13 (SP) D. R14 (LR)

1. `MOV R0, R1` 指令在Cortex-M和8051中的可用性是？ A. 两者都允许 B. 两者都禁止 C. Cortex-M允许，8051禁止 D. Cortex-M禁止，8051允许

1. 要将一个32位的立即数加载到寄存器R0，通常需要哪两条指令配合使用？ A. `MOV` 和 `ADD` B. `LDR` 和 `STR` C. `MOVW` 和 `MOVT` D. `MVN` 和 `AND`

1. 指令 `MVN R0, R1` 的功能是？ A. 将R1的值传送到R0 B. 将R1的值按位取反后传送到R0 C. 将R1和R0的值交换 D. 判断R1和R0的值是否相等

1. 要从内存地址 `[R1]` 处读取一个无符号字节(8位)到R0，应使用哪条指令？ A. `LDR R0, [R1]` B. `LDRH R0, [R1]` C. `LDRB R0, [R1]` D. `LDRSB R0, [R1]`

1. 指令 `LDR R0, [R1, #4]!` 的功能描述，正确的是？ A. 读取 `[R1+4]` 地址处的数据到R0，然后R1的值不变 B. 读取 `[R1]` 地址处的数据到R0，然后R1的值更新为 `R1+4` C. 读取 `[R1+4]` 地址处的数据到R0，然后R1的值更新为 `R1+4` D. 这是一个非法的指令格式

1. `STMIA R0!, {R1-R3}` 这条指令属于哪种操作？ A. 单寄存器加载 B. 单寄存器存储 C. 多寄存器加载 D. 多寄存器存储

1. 在 `STMIA R0!, {R1-R3}` 指令执行后，R0的值会如何变化？ A. 不变 B. 增加4 C. 增加8 D. 增加12

1. 指令 `B.EQ Label` 的含义是？ A. 如果Z标志位为0，则跳转到Label B. 如果Z标志位为1，则跳转到Label C. 无论如何都跳转到Label D. 这是一个函数调用指令

1. 要实现 `if (R0 == R1) R2 = R3;` 这样的条件执行，可以使用哪条指令？ A. `ITE EQ` B. `CMP R0, R1` 后跟 `MOVEQ R2, R3` C. `BEQ` D. 只能用跳转指令实现

1. Cortex-M的位段操作(Bit-banding)提供了一种什么性质的操作？ A. 可重入操作 B. 原子操作 C. 异步操作 D. 宏操作

1. STM32F4系列MCU中，配置GPIO端口方向模式的寄存器是？ A. `GPIOx_OTYPER` B. `GPIOx_PUPDR` C. `GPIOx_MODER` D. `GPIOx_IDR`

1. 要快速地将GPIOx端口的第5位置1，同时不影响其他位，最高效的操作是写入哪个寄存器？ A. `GPIOx_ODR` B. `GPIOx_BSRR` C. `GPIOx_MODER` D. `GPIOx_LCKR`

1. 在`GPIOx_BSRR`寄存器中，要对端口的第n位进行复位（清零），应该怎么做？ A. 对该寄存器的 `BSn` 位写1 B. 对该寄存器的 `BSn` 位写0 C. 对该寄存器的 `BRn` 位写1 D. 对该寄存器的 `BRn` 位写0

1. 以下哪个定时器是STM32F4系列中的高级控制定时器？ A. TIM2 B. TIM6 C. TIM8 D. TIM9

1. 在STM32的定时器中，用于设置PWM占空比的寄存器是？ A. 预分频寄存器 (PSC) B. 自动重装载寄存器 (ARR) C. 捕获/比较寄存器 (CCR) D. 控制寄存器 (CR)

1. 要实现无需CPU干预的内存与外设之间的数据传输，应使用哪个外设？ A. NVIC B. MPU C. DMA D. SysTick

1. 在C语言中，使用结构体指针来访问外设寄存器有什么优点？ A. 可以减少代码量 B. 编译速度更快 C. 将相关寄存器地址映射到结构体成员，使代码更具可读性和可移植性 D. 可以绕过内存保护

1. 在Cortex-M处理器中，执行 `BX LR` 指令通常用于什么目的？ A. 从函数中返回 B. 进入中断服务 C. 复位处理器 D. 无条件长跳转

1. Cortex-M的位段别名区(Bit-band Alias)的大小是多少？ A. 1MB B. 4MB C. 32MB D. 128MB

1. 如果要将R1左移2位后作为偏移量，与基址R2相加，从结果地址处加载一个字到R0，应该使用哪条指令？ A. `LDR R0, [R2, R1, LSL #2]` B. `LDR R0, [R2, #2]` C. `LDMIA R2, {R0, R1}` D. `LDR R0, [R2], R1, LSL #2`

1. **关于Cortex-M处理器的基本特性，下列描述错误的是？ (D)**
    - **解析：** Cortex-M是32位处理器，但除了能处理32位的字(word)，它还能高效处理8位的字节(byte)和16位的半字(half-word)。选项D描述不全面，是错误的。

1. **Cortex-M4处理器默认采用的数据存储顺序是？ (B)**
    - **解析：** 根据笔记，Cortex-M4采用小端模式(Little-Endian)。即一个多字节数据（如word）的低位字节存放在较低的内存地址处。

1. **在Cortex-M中，为了保证数据访问效率，一个半字(Half-word)变量的存放地址应该是？ (B)**
    - **解析：** Cortex-M采用对齐传输方式以提高效率。字节可以存放在任意地址，半字（16位=2字节）地址应为2的整数倍，字（32位=4字节）地址应为4的整数倍。

1. **Cortex-M3/M4处理器的指令流水线是几级？ (B)**
    - **解析：** 根据笔记，Cortex-M3/M4具有三级流水线，分别是取指、译码、执行。

1. **Cortex-M处理器支持的Thumb-2指令集，其指令长度是？ (C)**
    - **解析：** Thumb-2指令集的一大特点是它混合了16位和32位指令，兼顾了传统Thumb指令的高代码密度和ARM指令的高性能。

1. **在GPIO的复位状态上，Cortex-M与8051的主要区别是？ (C)**
    - **解析：** 根据笔记对比，Cortex-M的I/O口在复位时通常为高阻态（输入模式），而8051的I/O口复位后为高电平（P1/P2/P3）或需要上拉（P0）。

1. **Cortex-M处理器采用了哪种计算机体系结构？（B)**
    - **解析：** Cortex-M采用的是改进型哈佛结构。它有独立的指令总线和数据总线，可以同时取指和读写数据，提高了性能，但地址空间是统一的。
1. **关于Cortex-M的通用寄存器，下列描述正确的是？（D)**
    - **解析：** R0-R7被称为低寄存器，R8-R12被称为高寄存器。一些紧凑的16位Thumb指令只能访问低寄存器R0-R7。
1. **在Cortex-M处理器中，R13寄存器的专用功能是？（C)**
    - **解析：** R13被用作栈指针(Stack Pointer, SP)，用于PUSH和POP等堆栈操作。
1. **在未使用操作系统的情况下，Cortex-M处理器通常使用哪个栈指针？（A)**
    - **解析：** Cortex-M有两个栈指针，主栈指针(Main Stack Pointer, MSP)和进程栈指针(Process Stack Pointer, PSP)。MSP是默认使用的，用于处理异常和主程序；PSP通常在操作系统的任务上下文中使用。
1. **当Cortex-M处理器执行一个函数或子程序调用指令（如BL）时，返回地址会自动保存在哪个寄存器中？（B)**
    - **解析：** R14被用作链接寄存器(Link Register, LR)，在执行函数调用指令时，硬件会自动将下一条指令的地址（即返回地址）存入LR。
1. **关于Cortex-M中的程序计数器PC (R15)，下列说法与8051相比最显著的区别是？（C)**
    - **解析：** 8051的PC用户不可直接访问。而Cortex-M的PC (R15) 是可以被程序直接读写的，向PC写数据会直接引起程序跳转。
1. **在Cortex-M中，读取PC寄存器的值，得到的是？（C)**
    - **解析：** 由于三级流水线的存在，当读取PC时，取指部件已经预取了指令。因此，读PC返回的值是当前正在执行指令的地址+4。
1. **Cortex-M的程序状态寄存器xPSR不包含哪个部分？（D)**
    - **解析：** xPSR是三个状态寄存器的组合视图，包括APSR（应用）、IPSR（中断）和EPSR（执行）。没有DPSR（数据PSR）这个部分。
1. **APSR中的哪个标志位用于指示ALU运算结果为负数？（C)**
    - **解析：** N (Negative) 标志位。如果算术逻辑单元(ALU)的运算结果的最高位为1，则N位置1，表示结果为负数（在有符号数运算中）。
1. **在Cortex-M中，哪个寄存器保存了当前正在服务的中断号？（B)**
    - **解析：** IPSR (Interrupt PSR) 的低位部分用于存放当前激活的异常或中断的编号。
1. **EPSR中的T位如果被清零，将会发生什么？（C)**
    - **解析：** T位必须始终为1，以指示处理器处于Thumb状态。任何试图清除该位的操作（例如通过MSR指令）都会导致一个用法错误异常(Usage Fault)。
1. **Cortex-M的堆栈模型是？（D)**
    - **解析：** Cortex-M使用“向下生长的满栈”模型。向下生长意味着每次压栈，地址会减小；满栈意味着SP总是指向最后一个被压入堆栈的有效数据。
1. **在Cortex-M的堆栈操作中，执行一次PUSH操作，SP指针会如何变化？（D)**
    - **解析：** 因为是向下生长的满栈，所以在压入一个32位数时，SP会先自减4（为新数据腾出空间），然后再将数据存入SP指向的新地址。

1. **由于对齐要求，Cortex-M的堆栈指针SP的最低两位永远是？ (A)**
    - **解析：** 堆栈操作是基于32位（4字节）的，所以SP指针必须始终对齐到4字节边界。这意味着SP的值永远是4的倍数，其二进制表示的最低两位永远是00。

1. **Cortex-M的存储器映像中，通常用于存放程序的区域是？ (C)**
    - **解析：** 4GB的存储空间被划分为不同区域，地址较低的CODE区通常被设计用来存放程序代码（通常是Flash）。

1. **Cortex-M的存储器保护单元(MPU)的主要作用是？ (B)**
    - **解析：** MPU可以定义存储器区域的访问权限（如只读、可读写、禁止执行等），用于保护关键数据和代码，防止程序意外地破坏系统，在RTOS中尤为重要。

1. **Cortex-M处理器复位后，首先执行的操作是？ (C)**
    - **解析：** 根据复位流程，处理器复位后，硬件会首先从地址0x00000000处读取一个32位数，并将其加载到主堆栈指针MSP中。

1. **Cortex-M的向量表中，每个向量（地址）占多少位？ (C)**
    - **解析：** 向量表是一个存放地址的数组，每个地址都是32位的，指向对应的异常或中断服务程序的入口。

1. **向量表中的异常服务例程地址，其最低位必须为1，这是为了指示什么？ (C)**
    - **解析：** Cortex-M只支持Thumb状态。地址的最低位（LSB）被用作标志位，如果为1，表示目标地址的代码是Thumb指令。这个机制保证了在异常跳转时处理器能保持在正确的Thumb状态。

1. **嵌套向量中断控制器(NVIC)不具备以下哪个功能？ (D)**
    - **解析：** NVIC是中断控制器，负责中断的使能、屏蔽、优先级管理和仲裁，但它不执行具体的运算任务，运算是由CPU核心完成的。

1. **在NVIC中，如果两个中断的抢占优先级相同，响应优先级不同，当它们同时到达时，CPU会如何响应？ (B)**
    - **解析：** 当抢占优先级相同时，它们不能相互打断。此时由响应优先级（也称亚优先级或子优先级）决定响应顺序，响应优先级编号较小（即优先级较高）的那个中断会先被响应。

1. **在NVIC的优先级设置中，数值与其优先级的关系是？ (B)**
    - **解析：** Cortex-M的优先级编号越小，代表其优先级越高。例如，优先级0高于优先级1。

1. **在Cortex-M中，下列哪个属于内核异常，而不是外部中断(IRQ)？ (C)**
    - **解析：** SysTick定时器是集成在Cortex-M内核中的一个标准部件，其产生的中断属于系统异常，而不是由芯片制造商添加的外设中断(IRQ)。

1. **Cortex-M发生中断响应时，硬件自动压栈的寄存器不包括？ (C)**
    - **解析：** 硬件会自动将R0-R3, R12, LR, PC, xPSR这8个寄存器压栈。SP(R13)是用来执行压栈操作的指针，它自身的值会被更新，但不会被压入自己所指向的堆栈中。

1. `**MOV R0, R1**` **指令在Cortex-M和8051中的可用性是？ (C)**
    - **解析：** Cortex-M是RISC架构，具有丰富的通用寄存器，允许在寄存器之间直接传送数据。而8051是CISC架构，大部分操作需要通过累加器A中转，不允许直接在两个通用寄存器（如R0和R1）之间传送数据。

1. **要将一个32位的立即数加载到寄存器R0，通常需要哪两条指令配合使用？ (C)**
    - **解析：** 由于Thumb-2指令最长为32位，无法在一条指令中同时放下操作码和一个32位的立即数。因此，通常使用`MOVW`（Move Wide）加载低16位，再用`MOVT`（Move Top）加载高16位。

1. **指令 `**MVN R0, R1**` 的功能是？ (B)**
    - **解析：** `MVN`（Move Not）指令会将源操作数（R1）按位取反（求二进制反码），然后将结果传送到目标寄存器（R0）。

1. **要从内存地址 `**[R1]**` 处读取一个无符号字节(8位)到R0，应使用哪条指令？ (C)**
    - **解析：** `LDR`系列指令用于从内存加载数据。后缀`B`表示Byte（字节），`H`表示Half-word（半字）。无`S`表示无符号。因此`LDRB`是正确的。

1. **指令 `**LDR R0, [R1, #4]!**` 的功能描述，正确的是？ (C)**
    - **解析：** 这是立即数偏移（前序）寻址，并带有写回。`[R1, #4]`表示访问地址是R1+4。`!`后缀表示在操作完成后，要将计算出的新地址`R1+4`写回到R1寄存器中。

1. **`**STMIA R0!, {R1-R3}**` 这条指令属于哪种操作？ (D)**
    - **解析：** `STM`表示Store Multiple（多重存储）。`IA`表示Increase After（事后地址递增）。这条指令将寄存器列表`{R1-R3}`中的内容依次存入以R0为基址的内存中。

1. **在 `**STMIA R0!, {R1-R3}**` 指令执行后，R0的值会如何变化？ (D)**
    - **解析：** `R1-R3`共包含3个32位寄存器。`IA`表示事后地址增加，`!`表示写回。每存储一个32位寄存器，地址增加4。存储3个寄存器后，地址共增加`3 * 4 = 12`。

1. **指令 `**B.EQ Label**` 的含义是？ (B)**
    - **解析：** `B`是跳转指令。后缀`.EQ`（Equal）是条件码，它检查APSR中的Z（零）标志位。如果Z=1（表示前一条比较或算术指令结果为零），则执行跳转。

1. **要实现 `**if (R0 == R1) R2 = R3;**` 这样的条件执行，可以使用哪条指令？ (B)**
    - **解析：** 首先使用`CMP R0, R1`指令比较两个寄存器的值，该指令会根据比较结果更新APSR标志位（如果R0==R1，则Z=1）。然后使用带`.EQ`条件码的`MOV`指令`MOVEQ R2, R3`，该指令仅在Z=1时执行。

1. **Cortex-M的位段操作(Bit-banding)提供了一种什么性质的操作？ (B)**
    - **解析：** 位段操作的最大优点是它是一个“原子操作”。这意味着对单个位的读-修改-写操作是在一个总线周期内完成的，不会被其他总线主设备（如DMA）或中断打断，保证了位操作的完整性。

1. **STM32F4系列MCU中，配置GPIO端口方向模式的寄存器是？ (C)**
    - **解析：** `GPIOx_MODER` (GPIO port mode register) 用于配置每个引脚的工作模式，如输入、通用输出、复用功能或模拟模式。

1. **要快速地将GPIOx端口的第5位置1，同时不影响其他位，最高效的操作是写入哪个寄存器？ (B)**
    - **解析：** `GPIOx_BSRR` (GPIO port bit set/reset register) 提供了对端口位的原子置位和复位操作。向该寄存器的低16位（`BS0-BS15`）中的某一位写1，会原子地将对应的端口位置1，而不影响其他位，无需读-修改-写操作。

1. **在`**GPIOx_BSRR**`寄存器中，要对端口的第n位进行复位（清零），应该怎么做？ (C)**
    - **解析：** `BSRR`寄存器的高16位（`BR0-BR15`）用于复位。向`BRn`位写1，即可将对应的端口`Pxn`清零。

1. **以下哪个定时器是STM32F4系列中的高级控制定时器？ (C)**
    - **解析：** 根据笔记，TIM1和TIM8是高级控制定时器，功能最为强大，通常带有刹车输入、死区控制等功能，适用于电机控制。

1. **在STM32的定时器中，用于设置PWM占空比的寄存器是？ (C)**

- **解析：** PWM的占空比是通过比较计数器的当前值(CNT)和捕获/比较寄存器(CCR)的值来决定的。通过改变CCR的值就可以改变PWM的占空比。ARR决定周期，PSC决定计数频率。

1. **要实现无需CPU干预的内存与外设之间的数据传输，应使用哪个外设？ (C)**
   - **解析：** DMA (Direct Memory Access) 控制器就是为此设计的。它可以接管总线，在外设和内存之间直接传输数据，从而将CPU解放出来执行其他任务。

1. **在C语言中，使用结构体指针来访问外设寄存器有什么优点？ (C)**
   - **解析：** 将一个外设的所有相关寄存器定义在一个结构体中，然后将一个指向该外设基地址的结构体指针进行强制类型转换，就可以通过结构体成员名来访问各个寄存器。这使得代码逻辑清晰，易于理解和维护。

1. **在Cortex-M处理器中，执行 `**BX LR**` 指令通常用于什么目的？ (A)**
   - **解析：** `BX`（Branch and Exchange）指令会跳转到寄存器中给出的地址，并根据地址最低位切换状态（在此处保持Thumb状态）。由于LR中保存了函数调用的返回地址，因此`BX LR`是标准的从函数返回的指令。

1. **Cortex-M的位段别名区(Bit-band Alias)的大小是多少？ (C)**
   - **解析：** 1MB的位段区中的每一位，在32MB的位段别名区中都有一个对应的字地址。因此别名区的大小是32MB。

1. **如果要将R1左移2位后作为偏移量，与基址R2相加，从结果地址处加载一个字到R0，应该使用哪条指令？ (A)**
   - **解析：** 这是典型的寄存器偏移寻址模式。`LDR R0, [R2, R1, LSL #2]`完整地描述了这个操作：基址在R2，偏移量是R1逻辑左移2位（`LSL #2`），从`R2 + (R1 << 2)`地址处加载数据到R0。

EviterLesRoses

1. Cortex-M是32位RISC处理器，这意味着其数据总线、内部寄存器和指令长度都固定为32位。

1. Cortex-M4处理器采用小端模式，这意味着一个32位数据`0x12345678`存放在地址`0x20000000`时，地址`0x20000003`处存放的是数据`0x12`。

1. 为了访问效率，Cortex-M处理器要求所有类型的数据（字节、半字、字）都必须进行地址对齐。

1. Cortex-M的三级流水线在遇到跳转指令时，需要清空流水线，这会导致性能损失。

1. Cortex-M的GPIO引脚在上电复位后，其状态与8051单片机一样，都为高电平输出状态。

1. Cortex-M采用改进型哈佛结构，其程序总线和数据总线在硬件上是分离的，但地址空间是统一的。

1. 所有16位和32位的Thumb-2指令都可以无差别地访问R0-R12所有通用寄存器。

1. 在Cortex-M中，主栈指针MSP和进程栈指针PSP在物理上是两个完全独立的寄存器。

1. 执行`BL Label`指令后，返回地址被存入R14（LR），此时LR中存放的是`Label`的地址。

1. `MOV PC, LR`这条指令可以实现从子程序返回，其效果与`BX LR`完全相同。

1. 读取PC寄存器的值，得到的是当前正在执行指令的地址。

1. xPSR寄存器由APSR、IPSR和EPSR三部分组成，程序员可以通过`MSR`指令一次性修改xPSR中的所有位。

1. IPSR寄存器中的数值为0时，表示当前处理器不处于任何异常或中断服务模式下。

1. APSR中的S标志位（饱和标志）与N、Z、C、V标志位一样，可以作为条件转移指令的判断依据。

1. 对字变量`LDR R0, [R1]`和半字变量`LDRH R0, [R1]`进行非对齐访问时，处理器会产生一个硬件错误（HardFault）。

1. Cortex-M的堆栈是向上生长的满栈模型，即SP指针指向栈顶最后一个有效数据，并且每次压栈时SP地址会增加。

1. 执行指令`PUSH {R0, R1}`后，SP的值会减8，并且R0被存放在比R1更高的地址上。

1. Cortex-M处理器复位后，会首先从地址0x00000004处读取复位向量，然后才从0x00000000处加载MSP的初始值。

1. 向量表中的所有地址的最低位（LSB）必须为0，以表示处理器处于ARM状态。

1. 中断服务程序的入口地址就是向量表中对应中断号位置存放的32位数值。

1. NVIC中，优先级编号为3的中断可以抢占正在执行的优先级编号为5的中断。

1. 如果两个中断的抢占优先级和响应优先级都完全相同，当它们同时请求中断时，中断号较小的那个会得到响应。

1. Cortex-M响应中断时，硬件会自动压栈8个寄存器，这个过程不会被更高优先级的中断打断。

1. 指令`MOVS R0, R1`在传送数据的同时，会根据R1的值更新APSR中的N和Z标志位。

1. 执行`MOVW R0, #0x1234`后，R0的值为`0x00001234`；紧接着执行`MOVT R0, #0x5678`后，R0的值将变为`0x56781234`。

1. `LDRSB R0, [R1]`指令会从R1指向的地址加载一个字节，并将其进行零扩展到32位后存入R0。

1. 指令`STR R0, [R1], #4`是一个后序变址的存储指令，它会先将R0的值存入R1指向的地址，然后将R1的值加4。

1. 多重存储指令`STMDB R0!, {R1-R4}`中，`DB`表示在存储之前先将地址递减（Decrease Before）。

1. `B.LT Label`指令的跳转条件是APSR中的N标志位和V标志位不相等。

1. `ITTEE EQ`指令块允许在条件相等时，顺序执行两条Then指令，在条件不等时，顺序执行两条Else指令。

1. Cortex-M的位段操作（Bit-banding）本质上是通过软件查表和计算来实现对单个位的原子访问。

1. 在STM32中，要将一个GPIO引脚配置为复用功能模式，需要设置`GPIOx_MODER`寄存器中对应的两位为`10`。

1. 向`GPIOx_BSRR`寄存器的`BR1`位写1，可以使`GPIOx_ODR`寄存器的第1位清零。

1. STM32中的基本定时器（如TIM6, TIM7）也具备PWM输出和输入捕获功能。

1. 在配置PWM模式时，定时器的周期由预分频寄存器(PSC)的值决定，占空比由自动重装载寄存器(ARR)的值决定。

1. DMA控制器在进行一次数据传输前，必须由CPU设置源地址、目标地址和传输数量，一旦启动后，整个数据块的传输过程无需CPU干预。

1. 在C语言中，定义一个指向外设基地址的结构体指针，可以直接通过`->`操作符来读写该外设的所有寄存器。

1. Cortex-M处理器从异常返回时，必须执行`RETI`指令。

1. 1MB的位段区可以映射到32MB的位段别名区，这意味着对别名区的一个字节地址的访问，会映射到对位段区中一个特定位的操作。

1. `BX R0`指令执行时，如果R0的最低位是0，处理器将尝试切换到ARM状态并引发一个用法错误异常。

1. `CMP R0, #0`指令执行后，如果R0的值为0，则APSR中的Z标志位会被置1。

1. 链接寄存器LR (R14) 除了用于保存函数返回地址，在异常发生时，它会被硬件赋予一个特殊的EXC_RETURN值，用于指示异常返回时的处理器状态。

1. 进程栈指针PSP只能在特权模式下被修改。

1. 指令`LDR R0, =0x12345678`是一条伪指令，汇编器会将其转换为一条`MOV`指令或一条`LDR`指令从文字池中加载数据。

1. `STRH R0, [R1, R2]`指令表示将R0的低16位存储到地址(R1+R2)处。

1. SysTick定时器是一个24位的递减计数器，当它从1减到0时，可以产生一个SysTick异常。

1. MPU（内存保护单元）可以把内存划分为多个区域，并为每个区域独立设置访问权限，但它不能防止DMA控制器对受保护区域的非法访问。

1. 在一个抢占优先级高于当前执行代码的中断发生时，硬件压栈后，会立即从向量表取址并跳转，这个过程不存在流水线停顿。

1. 即使一个GPIO端口被配置为输出模式，程序依然可以读取其`IDR`寄存器来获取该引脚的实际电平状态。

1. 使用`__attribute__((packed))`关键字定义的C语言结构体，可以确保其成员变量在内存中是连续存放的，没有为了对齐而产生的填充字节，但这可能会导致非对齐访问，从而降低Cortex-M的访问性能。

EviterLesRoses

1. **Cortex-M是32位RISC处理器，这意味着其数据总线、内部寄存器和指令长度都固定为32位。**
    1. **答案：× (错误)**
    2. **解析：** 虽然数据总线和寄存器是32位的，但Cortex-M支持的Thumb-2指令集是变长指令集，包含了16位和32位两种长度的指令，并非固定的32位。

1. **Cortex-M4处理器采用小端模式，这意味着一个32位数据`**0x12345678**`存放在地址`**0x20000000**`时，地址`**0x20000003**`处存放的是数据`**0x12**`。**
    1. 
    2. **答案：√ (正确)**
    3. 
    4. **解析：** 小端模式(Little-Endian)指数据的低位字节存放在内存的低地址处，高位字节存放在高地址处。对于`0x12345678`，`78`是最低位字节，`12`是最高位字节。因此，存放地址从低到高依次为`0x78`(0x20000000)、`0x56`(0x20000001)、`0x34`(0x20000002)、`0x12`(0x20000003)。

1. **为了访问效率，Cortex-M处理器要求所有类型的数据（字节、半字、字）都必须进行地址对齐。**
    1. **答案：× (错误)**
    2. **解析：** 字节（8位）访问没有对齐要求，可以在任意地址进行。只有半字（16位）和字（32位）访问才建议进行地址对齐（地址分别是2和4的倍数）以获得最佳性能。部分Cortex-M核支持非对齐访问，但会牺牲性能。

1. **Cortex-M的三级流水线在遇到跳转指令时，需要清空流水线，这会导致性能损失。**
    1. **答案：√ (正确)**
    2. **解析：** 跳转指令（如B, BL, BX）会改变程序的正常执行顺序，导致流水线中已经取指和译码的指令变为无效，必须被清空（Flush），然后从新的目标地址重新开始取指，这会引入几个时钟周期的延迟。

1. **Cortex-M的GPIO引脚在上电复位后，其状态与8051单片机一样，都为高电平输出状态。**
    1. **答案：× (错误)**
    2. **解析：** 根据笔记，Cortex-M的GPIO在复位后通常被配置为高阻态输入模式，以防止对外部电路产生不确定的驱动。而8051的I/O口（P1-P3）复位后是高电平状态。

1. **Cortex-M采用改进型哈佛结构，其程序总线和数据总线在硬件上是分离的，但地址空间是统一的。**
    1. **答案：√ (正确)**
    2. **解析：** 这是改进型哈佛结构的典型特征。分离的总线允许处理器同时进行取指和数据访问，提高了性能。而统一的4GB地址空间意味着程序代码、数据和外设都在同一个地址映射中，可以通过地址方便地访问。

1. **所有16位和32位的Thumb-2指令都可以无差别地访问R0-R12所有通用寄存器。**
    1. **答案：× (错误)**
    2. **解析：** 这是一个重要的区别。许多紧凑的16位Thumb指令只能访问低寄存器组（R0-R7），而32位指令则可以访问所有通用寄存器（R0-R12）。

1. **在Cortex-M中，主栈指针MSP和进程栈指针PSP在物理上是两个完全独立的寄存器。**
    1. **答案：√ (正确)**
    2. **解析：** 处理器内部确实存在两个物理的栈指针寄存器。在任何时刻，只有一个是当前活动的SP，具体是哪个由CONTROL寄存器控制。这种设计是为了在操作系统环境中安全地隔离内核和用户任务的堆栈。

1. **执行 `**BL Label**` 指令后，返回地址被存入R14（LR），此时LR中存放的是 `**Label**` 的地址。**
    1. 
    2. **答案：× (错误)**
    3. 
    4. **解析：** `BL` 是带链接的跳转指令。LR中存放的是**下一条指令**的地址，即 `BL` 指令执行完毕后应该返回的地方，而不是跳转目标 `Label` 的地址。

1. **`**MOV PC, LR**` 这条指令可以实现从子程序返回，其效果与 `**BX LR**` 完全相同。**
    1. 
    2. **答案：× (错误)**
    3. 
    4. **解析：** `MOV PC, LR` 可以实现跳转，但它不等同于 `BX LR`。`BX` 指令会检查LR最低位来判断目标代码是ARM还是Thumb状态，而 `MOV` 指令不会，它仅仅是把地址值传给PC。在Cortex-M中，必须保持Thumb状态，因此标准的返回指令是 `BX LR` 或 `POP {PC}`。

1. **读取PC寄存器的值，得到的是当前正在执行指令的地址。**
    1. **答案：× (错误)**
    2. **解析：** 由于三级流水线的存在，当一条指令正在“执行”阶段时，PC已经指向了“取指”阶段的指令。因此，读取PC得到的值是“当前执行指令的地址 + 4”。

1. **xPSR寄存器由APSR、IPSR和EPSR三部分组成，程序员可以通过 `**MSR**` 指令一次性修改xPSR中的所有位。**
    1. **答案：× (错误)**
    2. **解析：** 程序员只能访问和修改APSR中的标志位，以及通过特定指令影响IPSR和EPSR。例如，EPSR中的T位是只读的，且必须为1，不能被修改。

1. **IPSR寄存器中的数值为0时，表示当前处理器不处于任何异常或中断服务模式下。**
    1. **答案：√ (正确)**
    2. **解析：** IPSR（中断程序状态寄存器）用于记录当前激活的异常编号。当没有异常或中断在服务时，其值为0，这被称为线程模式（Thread Mode）。

1. **APSR中的S标志位（饱和标志）与N、Z、C、V标志位一样，可以作为条件转移指令的判断依据。**
    1. **答案：× (错误)**
    2. **解析：** 根据笔记，S（Saturation）标志位用于指示饱和运算的状态，但它不作为条件转移的依据。条件转移只依赖于N, Z, C, V四个标志位。

1. **对字变量`**LDR R0, [R1]**`和半字变量`**LDRH R0, [R1]**`进行非对齐访问时，处理器会产生一个硬件错误（HardFault）。**
    1. **答案：× (错误)**
    2. **解析：** 不一定。这取决于处理器的配置。如果使能了非对齐访问，处理器会通过多次总线访问来完成操作，只是会降低性能。如果禁止了非对齐访问，才会产生用法错误（UsageFault），如果用法错误也被禁止，它可能会升级为硬件错误。

1. **Cortex-M的堆栈是向上生长的满栈模型，即SP指针指向栈顶最后一个有效数据，并且每次压栈时SP地址会增加。**
    1. **答案：× (错误)**
    2. **解析：** Cortex-M的堆栈是**向下生长**的满栈模型。向下生长意味着每次压栈，SP的地址会**减小**。

1. **执行指令`**PUSH {R0, R1}**`后，SP的值会减8，并且R0被存放在比R1更高的地址上。**
    1. 
    2. **答案：√ (正确)**
    3. 
    4. **解析：** `PUSH`两个32位寄存器，SP会减`2 * 4 = 8`。在ARM架构中，无论寄存器编号大小，总是将编号较小的寄存器存放在较低的地址。因此，R0存放在新的`SP`地址处，R1存放在`SP+4`地址处。所以R0的地址低于R1的地址。

1. **Cortex-M处理器复位后，会首先从地址0x00000004处读取复位向量，然后才从0x00000000处加载MSP的初始值。**
    1. 
    2. **答案：× (错误)**
    3. 
    4. **解析：** 顺序反了。根据复位流程图，处理器复位后，**首先**从地址`0x00000000`处读取MSP的初始值，**然后**才从`0x00000004`处读取复位向量（即程序的入口地址）。

1. **向量表中的所有地址的最低位（LSB）必须为0，以表示处理器处于ARM状态。**
    1. **答案：× (错误)**
    2. **解析：** 正好相反。Cortex-M只在Thumb状态下运行，因此向量地址的最低位**必须为1**，用来指示处理器跳转后应保持在Thumb状态。

1. **中断服务程序的入口地址就是向量表中对应中断号位置存放的32位数值。**
    1. **答案：× (错误)**

2. **解析：** 这是一个细微但重要的点。向量表中存放的32位数值，其最低位（bit 0）是Thumb状态指示位（必须为1），真正的入口地址是这个数值**将bit 0清零后**的结果。

1. **NVIC中，优先级编号为3的中断可以抢占正在执行的优先级编号为5的中断。**
    1. **答案：** √ **(正确)**
    2. **解析：** 在Cortex-M的NVIC中，优先级的数值越小，代表其优先级越高。因此，优先级3高于优先级5，可以抢占（打断）正在执行的优先级5的中断服务程序。

1. **如果两个中断的抢占优先级和响应优先级都完全相同，当它们同时请求中断时，中断号较小的那个会得到响应。**
    1. **答案：** √ **(正确)**
    2. **解析：** 当所有可配置的优先级都相同时，最终的仲裁依据是它们在向量表中的位置，即固定的中断号。中断号较小的中断具有天然的较高优先级。

1. **Cortex-M响应中断时，硬件会自动压栈8个寄存器，这个过程不会被更高优先级的中断打断。**
    1. **答案：** √ **(正确)**
    2. **解析：** 硬件压栈过程是原子性的，一旦开始，就必须完成，以保证堆栈的完整性。即使在压栈过程中有更高优先级的中断请求，也必须等待压栈完成后才能响应。

1. **指令 `**MOVS R0, R1**` 在传送数据的同时，会根据R1的值更新APSR中的N和Z标志位。**
    1. 
    2. **答案：** √ **(正确)**
    3. 
    4. **解析：** 指令后缀 `S` 的作用就是要求指令在执行后更新APSR中的条件标志位。`MOVS` 会根据传送的数据（即R1的值）来设置N（是否为负）和Z（是否为零）标志。

1. **执行 `**MOVW R0, #0x1234**` 后，R0的值为 `**0x00001234**`；紧接着执行 `**MOVT R0, #0x5678**` 后，R0的值将变为 `**0x56781234**`。**
    1. 
    2. **答案：** √ **(正确)**
    3. 
    4. **解析：** `MOVW`（Move Wide）用于加载一个16位的立即数到寄存器的低16位，同时将高16位清零。`MOVT`（Move Top）用于加载一个16位的立即数到寄存器的高16位，**且不影响低16位**。两者配合可以加载任意32位立即数。

1. **`**LDRSB R0, [R1]**` 指令会从R1指向的地址加载一个字节，并将其进行零扩展到32位后存入R0。**
    1. 
    2. **答案：** × **(错误)**
    3. 
    4. **解析：** 后缀 `SB` 代表 `Signed Byte`（有符号字节）。该指令加载一个字节后，会进行**符号位扩展**到32位。即如果加载的字节最高位是1（负数），则R0的高24位会被填充为1；如果最高位是0，则填充为0。零扩展对应的指令是 `LDRB`。

1. **指令 `**STR R0, [R1], #4**` 是一个后序变址的存储指令，它会先将R0的值存入R1指向的地址，然后将R1的值加4。**
    1. 

2. **答案：√ (正确)**
3. 
4. **解析：** 这是后序变址寻址。操作`[R1]`先执行，即在存储时使用R1当前的值作为地址。操作完成后，地址指针R1再更新（`R1 = R1 + 4`）。

1. **多重存储指令`**STMDB R0!, {R1-R4}**`中，`**DB**`表示在存储之前先将地址递减（Decrease Before）。**
    1. 
    2. **答案：√ (正确)**
    3. 
    4. **解析：** `DB`是`Decrease Before`的缩写。这意味着在存储每个寄存器之前，基址寄存器R0的值会先减4。`!`表示操作完成后，R0的最终值会写回到R0寄存器中。

1. `**B.LT Label**`**指令的跳转条件是APSR中的N标志位和V标志位不相等。**
    1. 
    2. **答案：√ (正确)**
    3. 
    4. **解析：** `LT` (Less Than) 条件码用于有符号数比较。跳转的条件是 `N != V`（负数标志不等于溢出标志）。

1. `**ITTEE EQ**`**指令块允许在条件相等时，顺序执行两条Then指令，在条件不等时，顺序执行两条Else指令。**
    1. 
    2. **答案：√ (正确)**
    3. 
    4. **解析：** `IT`指令块用于条件执行。`T`代表Then，`E`代表Else。`ITTEE`表示第一条指令是Then，第二、三条是Else，第四条是Else。如果条件`EQ`（相等）满足，则执行第一条指令，跳过后面三条；如果不满足，则跳过第一条，顺序执行后面三条。

1. **Cortex-M的位段操作（Bit-banding）本质上是通过软件查表和计算来实现对单个位的原子访问。**
    1. **答案：× (错误)**
    2. **解析：** 位段操作是由**处理器硬件**自动完成的。当CPU访问位段别名区的特定字地址时，总线硬件会自动将其转换为对位段区中对应位的读-修改-写操作，这个过程对软件是透明的。

1. **在STM32中，要将一个GPIO引脚配置为复用功能模式，需要设置`**GPIOx_MODER**`寄存器中对应的两位为`**10**`。**
    1. 
    2. **答案：√ (正确)**
    3. 
    4. **解析：** 根据笔记，`GPIOx_MODER`寄存器中，每个引脚由两位控制。`00`=输入，`01`=通用输出，`10`=复用功能，`11`=模拟模式。

1. **向`**GPIOx_BSRR**`寄存器的`**BR1**`位写1，可以使`**GPIOx_ODR**`寄存器的第1位清零。**
    1. 
    2. **答案：√ (正确)**
    3. 

4. **解析：** `BSRR`的高16位（`BR15-BR0`）用于复位（清零）对应的端口位。这是一个原子操作，比读-修改-写`ODR`寄存器更高效、更安全。

1. **STM32中的基本定时器（如TIM6, TIM7）也具备PWM输出和输入捕获功能。**
    1. **答案：× (错误)**
    2. **解析：** 基本定时器功能最简单，主要用于提供基本的定时和触发DAC等。它们没有捕获/比较通道，因此不具备PWM输出或输入捕获功能。这些功能由通用定时器和高级定时器提供。

1. **在配置PWM模式时，定时器的周期由预分频寄存器(PSC)的值决定，占空比由自动重装载寄存器(ARR)的值决定。**
    1. **答案：× (错误)**
    2. **解析：** 关系说反了。定时器的**周期**由自动重装载寄存器(ARR)的值决定（CNT计数到ARR的值为一个周期）。**占空比**由捕获/比较寄存器(CCR)与ARR值的比例决定。预分频寄存器(PSC)决定了CNT计数一次的时钟周期。

1. **DMA控制器在进行一次数据传输前，必须由CPU设置源地址、目标地址和传输数量，一旦启动后，整个数据块的传输过程无需CPU干预。**
    1. **答案：√ (正确)**
    2. **解析：** 这正是DMA的工作原理。CPU进行初始化配置后，DMA控制器就可以“接管”总线，在内存和外设之间搬运数据，直到传输完成产生中断，从而将CPU解放出来。

1. **在C语言中，定义一个指向外设基地址的结构体指针，可以直接通过`**>**`操作符来读写该外设的所有寄存器。**
    1. 
    2. **答案：√ (正确)**
    3. 
    4. **解析：** 这是STM32固件库和HAL库中广泛使用的编程技巧。通过将寄存器映射到结构体成员，可以用`peripheral_ptr->REGISTER_NAME`的形式访问寄存器，代码可读性强且结构清晰。

1. **Cortex-M处理器从异常返回时，必须执行`**RETI**`指令。**
    1. 
    2. **答案：× (错误)**
    3. 
    4. **解析：** `RETI`是8051单片机从中断返回的指令。Cortex-M从异常（包括中断）返回是通过向LR（链接寄存器）加载特殊的EXC_RETURN值，然后通过`BX LR`或`POP {PC}`等指令实现的。

1. **1MB的位段区可以映射到32MB的位段别名区，这意味着对别名区的一个字节地址的访问，会映射到对位段区中一个特定位的操作。**
    1. **答案：× (错误)**
    2. **解析：** 概念有误。是对别名区的一个**字（Word）地址**的访问，会映射到位段区中一个特定位的操作。因为对位的操作（置1或清0）是通过读写一个完整的字来实现的。

1. **`**BX R0**`指令执行时，如果R0的最低位是0，处理器将尝试切换到ARM状态并引发一个用法错误异常。**
    1. **答案：√ (正确)**
    2. **解析：** Cortex-M处理器只支持Thumb状态，因此跳转的目标地址最低位必须为1。如果尝试跳转到一个最低位为0的地址，硬件会认为这是一个非法的状态切换尝试，从而触发UsageFault异常。

1. **`**CMP R0, #0**` 指令执行后，如果R0的值为0，则APSR中的Z标志位会被置1。**
    1. 
    2. **答案：√ (正确)**
    3. 
    4. **解析：** `CMP` 指令通过执行减法来比较（但不保存结果），并根据减法结果更新标志位。如果 `R0 - 0` 的结果为0，则Z（零）标志位被置1。

1. **链接寄存器LR (R14) 除了用于保存函数返回地址，在异常发生时，它会被硬件赋予一个特殊的EXC_RETURN值，用于指示异常返回时的处理器状态。**
    1. 
    2. **答案：√ (正确)**
    3. 
    4. **解析：** 这是一个高级知识点。进入异常时，硬件会将LR设置为一个特殊的值（如 `0xFFFFFFF9`），这个值的高位是 `F`，低4位则包含了异常返回后应该恢复的处理器状态信息（例如返回到哪个堆栈，返回到特权模式还是用户模式等）。

1. **进程栈指针PSP只能在特权模式下被修改。**
    1. **答案：√ (正确)**
    2. **解析：** 对CONTROL寄存器（用于切换MSP/PSP）和PSP寄存器本身的直接写操作都属于系统级控制，必须在特权模式下才能执行，以保证操作系统的安全和稳定。

1. **指令 `**LDR R0, =0x12345678**` 是一条伪指令，汇编器会将其转换为一条 `**MOV**` 指令或一条 `**LDR**` 指令从文字池中加载数据。**
    1. 
    2. **答案：√ (正确)**
    3. 
    4. **解析：** `LDR Rd, =const` 是汇编器提供的伪指令，用于方便地加载一个32位常量。汇编器会根据常量的大小智能地选择最优的实现方式：如果常量可以用 `MOV` / `MOVW` 指令表示，就用它们；如果不行，汇编器会在代码段附近生成一个“文字池（Literal Pool）”存放这个常量，然后用一条PC相对的 `LDR` 指令去加载它。

1. **`**STRH R0, [R1, R2]**` 指令表示将R0的低16位存储到地址(R1+R2)处。**
    1. 
    2. **答案：√ (正确)**
    3. 
    4. **解析：** `STRH` 是存储半字（Half-word, 16位）的指令。寻址方式是基址(R1)加寄存器偏移(R2)。该指令会将R0的低16位内容，存储到由R1和R2内容相加得到的内存地址处。

1. **SysTick定时器是一个24位的递减计数器，当它从1减到0时，可以产生一个SysTick异常。**
    1. **答案：√ (正确)**
    2. **解析：** SysTick是Cortex-M内核标配的系统定时器，它是一个24位的递减计数器，非常适合用作操作系统的时钟节拍。当计数到0时，会触发一个SysTick异常（中断），并自动重装载初值。

1. **MPU（内存保护单元）可以把内存划分为多个区域，并为每个区域独立设置访问权限，但它不能防止DMA控制器对受保护区域的非法访问。**

    1. **答案：× (错误)**
    2. **解析：** MPU的保护是针对总线上的所有主设备（Bus Master）的，包括CPU核心和DMA控制器。因此，如果DMA试图向一个被MPU配置为只读的区域写入数据，同样会触发内存管理错误（MemManage Fault）。

1. **在一个抢占优先级高于当前执行代码的中断发生时，硬件压栈后，会立即从向量表取址并跳转，这个过程不存在流水线停顿。**
    1. **答案：× (错误)**
    2. **解析：** 任何改变程序控制流的操作，包括中断引发的强制跳转，都会导致流水线被清空和重新填充，从而产生延迟。虽然中断响应很快，但流水线停顿是不可避免的。

1. **即使一个GPIO端口被配置为输出模式，程序依然可以读取其 `**IDR**` 寄存器来获取该引脚的实际电平状态。**
    1. 
    2. **答案：√ (正确)**
    3. 
    4. **解析：** `ODR`（Output Data Register）是输出数据锁存器，决定了你要输出的电平。而`IDR`（Input Data Register）则总是反映该I/O引脚**物理上**的实际电平状态。这在检测短路或总线冲突等场景下非常有用。

1. **使用 `**__attribute__((packed))**` 关键字定义的C语言结构体，可以确保其成员变量在内存中是连续存放的，没有为了对齐而产生的填充字节，但这可能会导致非对齐访问，从而降低Cortex-M的访问性能。**
    1. 
    2. **答案：√ (正确)**
    3. 
    4. **解析：** `packed` 属性强制编译器取消为了对齐而添加的填充字节，使得结构体紧凑。但如果结构体内的半字或字成员因此落在了非对齐的地址上，CPU在访问它们时就需要执行非对齐访问，这会比对齐访问花费更多的时钟周期，降低性能，甚至在禁止非对齐访问时引发异常。